Multimodal table scanning application

---

1. Concept Introduction

1.1 What is "Multimodal Table Scanning"?

Multimodal table scanning is a technology that uses image processing and artificial intelligence to identify and extract table information from images or PDF documents. It not only focuses on visual table structure recognition but also incorporates multimodal data such as text content and layout information to enhance table understanding. **Large Language Models (LLMs)** provide powerful semantic analysis capabilities to understand this extracted information. The two complement each other and enhance the intelligence of document processing.

1.2 Implementation Principle Overview

1. **Table Detection and Content Recognition**

- Utilizes computer vision technology to locate tables in documents and uses optical character recognition (OCR) technology to convert the text within the tables into an editable format.
- Utilizes deep learning methods to analyze table structure (row and column division, cell merging, etc.) and generate a structured data representation.

2. **Multimodal Fusion**

- Integrates visual information (such as table layout), text (OCR results), and any metadata (such as file type and source) to form a comprehensive data view. - Use a specially designed multimodal model (such as LayoutLM) to simultaneously process these different types of data, thereby more accurately understanding the table content and its contextual relationships.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `scan_table` function in this file defines the tool's execution flow, specifically how it constructs a prompt that returns a Markdown-formatted result.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan a table from an image and save the content as a Markdown file.
        从图像中扫描表格，并将内容保存为Markdown文件。

        :param args: Arguments containing the image path.
        :return: Dictionary with file path and content.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            # ... (路径检查和回退)

            # Construct a prompt asking the large model to recognize the table
and return it in Markdown format.
            # 构造提示，要求大模型识别表格并以Markdown格式返回。
            if self.node.language == 'zh':
                prompt = "请仔细分析这张图片，识别其中的表格，并将其内容以Markdown格式返
回。"
            else:
                prompt = "Please carefully analyze this image, identify the
table within it, and return its content in Markdown format."

            result = self.node.model_client.infer_with_image(image_path, prompt)

            # ... (从结果中提取Markdown文本)

            # Save the recognized content to a Markdown file. / 将识别出的内容保存到
Markdown文件。
            md_file_path = os.path.join(self.node.pkg_path, "resources_file",
"scanned_tables", f"table_{timestamp}.md")
            with open(md_file_path, 'w', encoding='utf-8') as f:
                f.write(table_content)

            return {
                "file_path": md_file_path,
                "table_content": table_content
            }
        # ... (错误处理)
```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file serves as the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface. / 统一的图像推理接口。"""
        # ... (准备消息)
        try:
            # 根据 self.llm_platform 的值，决定调用哪个具体实现
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... 调用通义模型的逻辑
                pass
            # ... (其他平台的逻辑)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The table scanning function is a typical application for converting unstructured image data into structured text data. Its core technology remains **guiding model behavior through prompt engineering**.

1. **Tools Layer (`tools_manager.py`)**:

- The `scan_table` function is the business process controller for this function. It receives an image containing a table as input.
- The key operation of this function is **building a targeted prompt**. This prompt directly instructs the large model to perform two tasks: 1. Recognize the table in the image. 2. Return the recognized content in Markdown format. This mandatory output format is key to achieving unstructured-to-structured conversion.
- After constructing the prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and the formatting instructions.
- After receiving the Markdown text returned from the model interface layer, it performs a file operation: writing the text content to a new `.md` file.
- Finally, it returns structured data containing the new file path and table contents.

2. **Model Interface Layer (`large_model_interface.py`)**:

- The `infer_with_image` function continues to serve as the unified "dispatching center." It receives the image and prompt from `scan_table` and dispatches the task to the correct backend model implementation based on the current system configuration (`self.llm_platform`).
- Regardless of the backend model, this layer handles the communication details with the specific platform, ensuring that the image and text data are sent correctly, and then returns the plain text (in this case, Markdown-formatted text) returned by the model to the tooling layer.

In summary, the general workflow for table scanning is: ToolsManager receives an image and constructs a command to convert the table in this image to Markdown. ToolsManager calls the model interface. model_interface packages the image and the command and sends it to the corresponding model platform according to the configuration. The model returns Markdown-formatted text. model_interface returns the text to ToolsManager. ToolsManager saves the text as a .md file and returns the result. This workflow demonstrates how to leverage the formatting capabilities of a large model as a powerful OCR (Optical Character Recognition) and data structuring tool.

3. Practical Operations

3.1 Configuring the Offline Large Model

3.1.1 Configuring the LLM Platform (yahboom.yaml)

This file determines which large model platform the model_service node loads as its primary language model.

1. **Open the file in the terminal**:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/Confirm** `llm_platform`:

```
model_service:                          #模型服务器节点参数 Model server node parameters
  ros__parameters:
    language: 'en'                      #大模型接口语言 Large Model Interface Language
    useolinetts: False                   #文字模式下此项无效，可忽略 This item is invalid in text mode and can be ignored

    # 大模型配置 Large model configuration
    llm_platform: 'ollama'              # 关键: 确保这里是 'ollama' Key: Make sure it's 'ollama'
    regional_setting : "international"
```

3.1.2 配置模型接口 (`large_model_interface.yaml`)

此文件定义了当平台被选为 `ollama` 时，具体使用哪个视觉模型。

1.在终端打开文件

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2.找到ollama相关的配置

```
#.....
## Offline Large Language Models
# Ollama Configuration
ollama_host: "http://localhost:11434" # Ollama server address
ollama_model: "llava" # Key: Replace this with the multimodal model you downloaded, such as "llava"
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Starting and Testing (Text Mode)

Note: Due to performance limitations on the Jetson Orin Nano 4GB, performance may be poor. To experience this feature, please refer to the corresponding section in <Online Large Model (Text Interaction)>.

1. **Prepare a table image file**:

Place a table image file to test in the following path:
`/home/jetson/yahboom_ws/src/largemodel/resources_file/scan_table`

Then name the image `test_table.jpg`

2. **Start the `largemodel` main program (text mode)**:

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py ••text_chat_mode:=true
```

3. **Send a text command**:
   Open another terminal and run the following command:

```
ros2 run text_chat text_chat
```

Then start typing: "Analyze the table."

4. **Observation Results**:
   In the first terminal running the main program, you will see log output indicating that the system received the command, called the `scan_table` tool, and completed the scan_table execution, saving the scanned information to a document.

This document can be found in the ~/yahboom_ws/src/largemodel/resources_file/scan_table directory.

4. Common Problems and Solutions

Problem 1: Incomplete recognition of table content or typos.

Solution:

1. **Image Quality**: Ensure that the input table image is clear, undistorted, and evenly lit. Poor image quality is the most common cause of recognition errors.
2. **Model Selection**: Models with larger parameters tend to perform better. Try switching to a more powerful recognition model.

Problem 2: "Table file not found" error

Solution:

1. **Check the path**: Ensure you have placed the table image file in the specified path, named it `test_table.jpg`, and have read permissions for the file.
2. **Image format**: Ensure the image file is not corrupted and is in .jpg format.